

```

1 #=====
2 # MIT 6.046J 代码精读 • PS5 - 增强跳跃表 (finger/rank)
3 # 源文件: skiplist.py (来自 GitHub mit-6046)
4 #=====
5
6 """PS5 Problem 5-1: skip list with FINGER-SEARCH and RANK-SEARCH.
7
8 A randomized skip list over distinct integer keys, augmented with per-forward-
9 pointer "span" counts (the number of level-0 nodes the pointer skips over) so we
10 can support:
11
12 * SEARCH / INSERT / DELETE          --  $O(\lg n)$  w.h.p. (standard);
13 * FINGER-SEARCH(x, k)                --  $O(\lg m)$  w.h.p.,  $m = 1 + |\text{rank}(x) - \text{rank}(k)|$ 
14                                     (part a): climb up from x until the key
15                                     is bracketed, then descend -- the number
16                                     of levels touched is  $O(\lg m)$ ;
17 * RANK-SEARCH(x, r)                  --  $O(\lg m)$  w.h.p.,  $m = r + 1$  (parts b,c):
18                                     walk forward/using spans by exactly r
19                                     positions from x.
20
21 Each node stores, per level, (forward_pointer, span) where span = number of
22 level-0 steps that pointer covers. Spans are what make rank arithmetic  $O(1)$  per
23 hop and give RANK-SEARCH its  $O(\lg m)$  bound. We keep -inf and +inf sentinels on
24 every level as the pset allows.
25 """
26 from __future__ import annotations
27
28 import random
29 from typing import List, Optional
30
31 NEG_INF = float("-inf")
32 POS_INF = float("inf")
33
34
35 class Node:
36     __slots__ = ("key", "forward", "span", "back")
37
38     def __init__(self, key, level: int):
39         self.key = key
40         self.forward: List[Optional["Node"]] = [None] * (level + 1)
41         self.span: List[int] = [0] * (level + 1) # span[i] = #level-0 nodes hop i covers
42         self.back: Optional["Node"] = None      # level-0 back pointer (for finger climb)
43
44     def level(self) -> int:
45         return len(self.forward) - 1
46
47
48 class SkipList:
49     def __init__(self, p: float = 0.5, max_level: int = 32, seed: Optional[int] = None):
50         self.p = p
51         self.max_level = max_level
52         self.rng = random.Random(seed)
53         self.head = Node(NEG_INF, max_level)
54         self.tail = Node(POS_INF, max_level)
55         for i in range(max_level + 1):
56             self.head.forward[i] = self.tail
57             # span from head to tail initially = 1 (only the +inf sentinel beyond level-0)
58             self.head.span[i] = 1
59         self.tail.back = self.head
60         self.level = 0 # current highest populated level
61         self.size = 0  # number of real keys (excludes sentinels)
62
63     # ----- helpers -----
64     def _random_level(self) -> int:
65         lvl = 0
66         while self.rng.random() < self.p and lvl < self.max_level:
67             lvl += 1
68         return lvl
69
70     # ----- SEARCH -----
71     def search(self, key) -> Optional[Node]:
72         x = self.head
73         for i in range(self.level, -1, -1):
74             while x.forward[i] is not None and x.forward[i].key < key:
75                 x = x.forward[i]
76         x = x.forward[0]
77         if x is not None and x.key == key:
78             return x
79         return None
80
81     # ----- INSERT (maintains spans, back pointers) -----
82     def insert(self, key) -> None:
83         update: List[Node] = [self.head] * (self.max_level + 1)
84         rank = [0] * (self.max_level + 1) # rank[i] = #level-0 steps from head to update[i]
85         x = self.head

```

```

86     for i in range(self.level, -1, -1):
87         if i != self.level:
88             rank[i] = rank[i + 1]
89         while x.forward[i] is not None and x.forward[i].key < key:
90             rank[i] += x.span[i]
91             x = x.forward[i]
92         update[i] = x
93     if x.forward[0] is not None and x.forward[0].key == key:
94         return # distinct keys assumed; ignore duplicate
95
96     lvl = self._random_level()
97     if lvl > self.level:
98         for i in range(self.level + 1, lvl + 1):
99             rank[i] = 0
100            update[i] = self.head
101            self.head.span[i] = self.size + 1 # head -> tail spans everything
102            self.level = lvl
103
104            new = Node(key, lvl)
105            for i in range(lvl + 1):
106                new.forward[i] = update[i].forward[i]
107                update[i].forward[i] = new
108                # span bookkeeping
109                new.span[i] = update[i].span[i] - (rank[0] - rank[i])
110                update[i].span[i] = (rank[0] - rank[i]) + 1
111            # levels above lvl: their pointers now skip one more node
112            for i in range(lvl + 1, self.level + 1):
113                update[i].span[i] += 1
114
115            # back pointers at level 0
116            new.back = update[0]
117            if new.forward[0] is not None:
118                new.forward[0].back = new
119            self.size += 1
120
121            # ----- DELETE -----
122            def delete(self, key) -> bool:
123                update: List[Node] = [self.head] * (self.max_level + 1)
124                x = self.head
125                for i in range(self.level, -1, -1):
126                    while x.forward[i] is not None and x.forward[i].key < key:
127                        x = x.forward[i]
128                    update[i] = x
129                target = x.forward[0]
130                if target is None or target.key != key:
131                    return False
132                for i in range(self.level + 1):
133                    if update[i].forward[i] is target:
134                        update[i].span[i] += target.span[i] - 1
135                        update[i].forward[i] = target.forward[i]
136                    else:
137                        update[i].span[i] -= 1
138                if target.forward[0] is not None:
139                    target.forward[0].back = target.back
140                # lower the list level while the top level is empty (head -> tail directly).
141                # The abandoned level's head span becomes head->tail = (#real nodes after
142                # this delete) + 1 (the +inf sentinel), keeping every level self-consistent.
143                while self.level > 0 and self.head.forward[self.level] is self.tail:
144                    self.head.span[self.level] = (self.size - 1) + 1 # = new size + tail
145                    self.level -= 1
146                self.size -= 1
147                return True
148
149            # ----- rank of a key (0-based over real keys) -----
150            def rank(self, key) -> int:
151                r = 0
152                x = self.head
153                for i in range(self.level, -1, -1):
154                    while x.forward[i] is not None and x.forward[i].key < key:
155                        r += x.span[i]
156                        x = x.forward[i]
157                return r # number of real keys strictly less than `key`
158
159            # ----- FINGER-SEARCH (part a) -----
160            def finger_search(self, x: Node, k) -> Node:
161                """Return the node holding key k, starting from level-0 node x.
162                Climb up until k is bracketed, then descend.  O(lg m) w.h.p."""
163                if x.key == k:
164                    return x
165                if k > x.key:
166                    # CLIMB x's own tower as high as still useful: stay at level i while the
167                    # forward pointer there does not overshoot k. This reaches level
168                    # O(lg m) because a level-i pointer spans ~2^i nodes.
169                    i = 0
170                    while (i < x.level()
171                        and x.forward[i + 1] is not None

```

```

172         and x.forward[i + 1].key <= k):
173             i += 1
174         # DESCEND / advance: a standard top-down skip-list search from (x, i).
175         # Terminates because each iteration either advances `node` forward
176         # (key strictly increases) or drops `i` by one.
177         node = x
178         while True:
179             if node.forward[i] is not None and node.forward[i].key <= k:
180                 node = node.forward[i]
181                 if node.key == k:
182                     return node
183             elif i > 0:
184                 i -= 1
185             else:
186                 raise KeyError(k)
187         else: # k < x.key : walk backwards along level 0 (m is small here)
188             node = x
189             while node is not None and node.key > k:
190                 node = node.back
191             if node is not None and node.key == k:
192                 return node
193             raise KeyError(k)
194
195     # ----- RANK-SEARCH (part c) -----
196     def rank_search(self, x: Node, r: int) -> Node:
197         """Return the node whose rank is rank(x)+r, using spans. O(lg m) w.h.p."""
198         if r == 0:
199             return x
200         remaining = r
201         node = x
202         i = 0
203         while remaining > 0:
204             # climb while the next span at level i fits within remaining
205             while i < node.level() and node.forward[i] is not None and node.span[i] <= remaining:
206                 i += 1
207             # descend to a level whose span fits
208             while i > 0 and (node.forward[i] is None or node.span[i] > remaining):
209                 i -= 1
210             if node.forward[i] is None or node.span[i] > remaining:
211                 # can only move at level 0
212                 node = node.forward[0]
213                 remaining -= 1
214             else:
215                 remaining -= node.span[i]
216                 node = node.forward[i]
217         return node
218
219     # ----- introspection -----
220     def to_list(self) -> List:
221         out = []
222         x = self.head.forward[0]
223         while x is not None and x.key != POS_INF:
224             out.append(x.key)
225             x = x.forward[0]
226         return out
227
228     def check_spans(self) -> None:
229         """Verify span[i] of every node equals the true #level-0 nodes covered."""
230         # index each real node by its level-0 position
231         pos = {}
232         idx = 0
233         x = self.head
234         while x is not None:
235             pos[id(x)] = idx
236             if x.key != POS_INF:
237                 idx += 1
238             x = x.forward[0]
239         # Validate spans on every ACTIVE level (0..self.level). Levels above
240         # self.level are inactive (never traversed: searches start at self.level)
241         # and their head/tail sentinel spans are intentionally left stale.
242         node = self.head
243         while node is not None:
244             top = min(node.level(), self.level)
245             for i in range(top + 1):
246                 fwd = node.forward[i]
247                 if fwd is None:
248                     continue
249                 covered = pos[id(fwd)] - pos[id(node)]
250                 assert node.span[i] == covered, (
251                     f"span mismatch at key {node.key} level {i}: "
252                     f"stored {node.span[i]} actual {covered}")
253             node = node.forward[0]
254

```